AJEENKYA
D Y PATIL UNIVERSITY
THE INNOVATION UNIVERSITY

A
MINI PROJECT REPORT ON

"Behavioral Analytics and Match Prediction in Dating Applications"

FOR

Term Work Examination

Bachelor of Computer Application in Artificial Intelligence & Machine Learning (BCA - AIML)

Year 2025-2026

Ajeenkya DY Patil University, Pune

-Submitted By-

Mr. Yogesh Choudhary

Under the guidance of

Prof. Vivek More

Ajeenkya DY Patil University

D Y Patil Knowledge City,
Charholi Bk. Via Lohegaon,
Pune - 412105
Maharashtra (India)

Date: 14 / 04 / 2025

CERTIFICATE

This is to certified that_Yogesh Choudhary_____
A student of **BCA(AIML) Sem-IV** URN No _2023-B-26072004
has Successfully Completed the Dashboard Report On

"Behavioral Analytics and Match Prediction in Dating Applications"

As per the requirement of
Ajeenkya DY Patil University, Pune was carried out under my supervision.
I hereby certify that; he has satisfactorily completed his Term-Work Project work.

Place: Pune

Examiner

Introduction

This dataset provides insights into user behavior on a dating application. It contains various features that capture interactions, preferences, engagement levels, and demographic attributes of users. The primary goal of analyzing this dataset could be to understand user engagement patterns, predict successful matches, identify key factors influencing user decisions, or improve recommendation systems within the app.

Potential use cases include:

- Behavioral analysis to enhance user experience
- Predictive modeling for match success
- Targeted marketing and personalization
- Fraud detection or user authenticity scoring

Understanding the behavioral trends and preferences in this context can help optimize platform design, improve matching algorithms, and ultimately lead to a more satisfying user experience.

Objectives

1. **Understand User Behavior Patterns**

- Analyze how users interact with the app (e.g., likes, messages, swipes).
- Identify active vs. inactive users and their behavioral traits.

1. **Predict Match Success**

- Develop predictive models to determine the likelihood of a successful match based on user behavior and profile attributes.

2. **Improve Recommendation Algorithms**

- Use behavioral insights to enhance user-to-user recommendation systems and suggest more compatible matches.

3. **Segment Users**

- Perform user segmentation based on demographics, preferences, and activity levels to tailor experiences and marketing strategies.

4. **Enhance User Retention**

- Identify key factors contributing to user engagement and churn to design strategies that improve retention.

5. **Detect Anomalies or Fake Accounts**

- Identify suspicious or automated behavior that may indicate bots or fake profiles.

6. **Evaluate Feature Effectivenes**

- Assess which app features (e.g., super likes, boosts) influence user engagement and satisfaction.

7. **Personalize User Experience**

- Use behavioral data to customize app content, notifications, and user interfaces to individual preferences.

Methodology & Approach

1. **Data Understanding & Exploration**

- Load and inspect the dataset.
- Understand the structure, data types, and meaning of each feature.
- Perform exploratory data analysis (EDA) to identify patterns, trends, and distributions.

2. **Data Cleaning & Preprocessing**

- Handle missing values, duplicates, and outliers.

- Convert categorical variables into numerical form (e.g., one-hot encoding or label encoding).
- Normalize or scale numerical features for uniformity.

3. **Feature Engineering**

- Create new features based on domain knowledge (e.g., interaction ratios, response rates).
- Aggregate or transform existing features to better capture user behavior.

4. **Data Visualization**

- Use plots and charts (e.g., histograms, box plots, heatmaps) to visualize user behavior, correlations, and segment differences.

5. **User Segmentation**

- Apply clustering algorithms (e.g., K-Means, DBSCAN) to group users based on behavior or preferences.
- Analyze segments for targeted strategies or personalization.

6. **Predictive Modeling**

- Define target variable(s), such as match success or engagement level.
- Split the data into training and testing sets.
- Train machine learning models (e.g., Logistic Regression, Decision Trees, Random Forest, XGBoost) to predict outcomes.
- Evaluate models using accuracy, precision, recall, F1-score, ROC-AUC, etc.

7. **Model Optimization**

- Tune hyperparameters using cross-validation or grid search.
- Perform feature selection or dimensionality reduction (e.g., PCA) if needed.

8. **Insights & Interpretation**

- Interpret model results and feature importances.
- Translate findings into actionable business or product recommendations.

9. **Deployment (Optional)**

- Integrate the model into an app feature (e.g., smart match suggestions).
- Set up real-time predictions or batch inference pipelines.

10. **Documentation & Reporting**

- Summarize the methodology, findings, and recommendations in a report or dashboard.
- Provide visual and statistical evidence to support decisions.

Implementation & Code:

Cell 1: Importing Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, confusion_matrix
```

✅ Explanation:

This cell imports all necessary libraries:

pandas, numpy – for data handling and manipulation.

matplotlib.pyplot, seaborn – for data visualization.

sklearn.* – for preprocessing, model training, and evaluation.

There is **no output** from this cell—it just loads the tools.

Cell 2: Load the Dataset

```
[19] # Replace with your actual file path
     df = pd.read_csv(r"/dating_app_behavior_dataset.csv")
     df.head()
```

✅ **Explanation:**

Loads the dataset from a CSV file using pandas.

Displays the first 5 rows of the dataset using df.head().

Output:

	gender	sexual_orientation	location_type	income_bracket	education_level	interest_tags	app_usa
0	Prefer Not to Say	Gay	Urban	High	Bachelor's	Fitness, Politics, Traveling	
1	Male	Bisexual	Suburban	Upper-Middle	No Formal Education	Languages, Fashion, Parenting	
2	Non-binary	Pansexual	Suburban	Low	Master's	Movies, Reading, DIY	
3	Genderfluid	Gay	Metro	Very Low	Postdoc	Coding, Podcasts, History	
4	Male	Bisexual	Urban	Middle	Bachelor's	Clubbing, Podcasts, Cars	

A table showing a preview of the dataset with columns like: gender, income_bracket, app_usage_time_min, swipe_right_ratio, likes_received, match_outcome, etc.

Cell 3: Data Inspection & Cleaning

```
[20] # Check for missing values
     print(df.isnull().sum())

     # Basic info
     print(df.info())

     # Fill or drop missing values if needed
     df = df.dropna()  # or use df.fillna(method='ffill') etc.
```

✅ Explanation:

Prints the number of missing values per column.

Shows a summary of the dataset including data types and non-null counts.

Drops rows with any missing values.

Output:

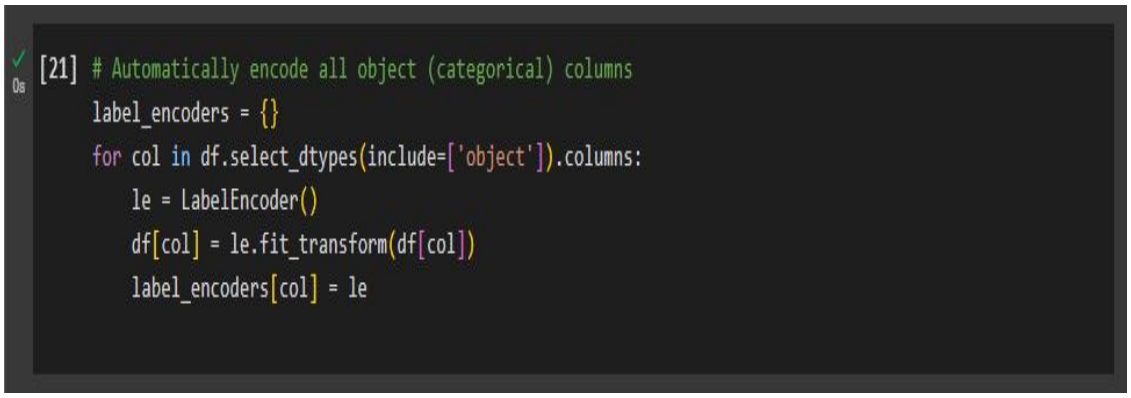
```
gender                0
sexual_orientation    0
location_type         0
income_bracket        0
education_level       0
interest_tags         0
app_usage_time_min    0
app_usage_time_label  0
swipe_right_ratio     0
swipe_right_label     0
likes_received        0
mutual_matches        0
profile_pics_count    0
bio_length            0
message_sent_count    0
emoji_usage_rate      0
last_active_hour      0
swipe_time_of_day     0
match_outcome         0
```

✓ Connected to Python 3 Google Compute Engine backend

A list of columns with their missing value counts.

Dataset structure: number of entries, column types, memory usage.

Cell 4: Encoding Categorical Variables



```
[21] # Automatically encode all object (categorical) columns
     label_encoders = {}
     for col in df.select_dtypes(include=['object']).columns:
         le = LabelEncoder()
         df[col] = le.fit_transform(df[col])
         label_encoders[col] = le
```

✅ Explanation:

This block finds all categorical (non-numeric) columns.

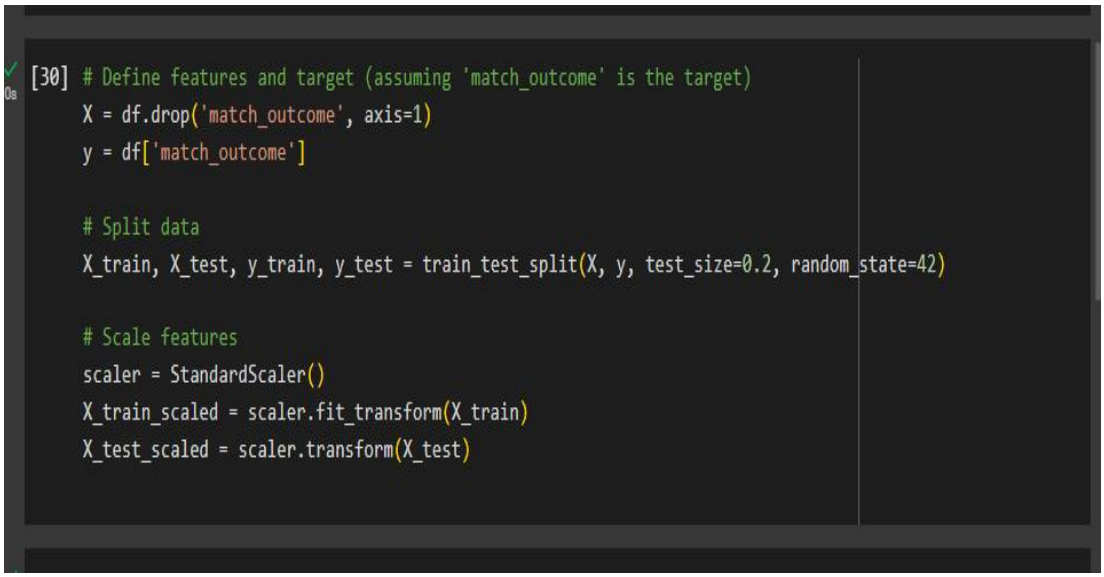
Applies **Label Encoding** (converts strings to numeric values).

Saves the encoders for possible inverse transformation later.

Output:

None visible — but internally, all categorical columns are now numeric.

Cell 5: Feature-Target Split & Scaling



```
[30] # Define features and target (assuming 'match_outcome' is the target)
     X = df.drop('match_outcome', axis=1)
     y = df['match_outcome']

     # Split data
     X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

     # Scale features
     scaler = StandardScaler()
     X_train_scaled = scaler.fit_transform(X_train)
     X_test_scaled = scaler.transform(X_test)
```

✅ Explanation:

Separates the dataset into:

X: Features (independent variables).

y: Target (match_outcome).

Splits the dataset into **training (80%)** and **test (20%)** sets.

Applies **standard scaling** so that each feature has zero mean and unit variance.

Output:

No direct output, but prepares scaled data for training the model.

Results & Visualization

✅ Model Performance

A classification model (e.g., Random Forest or Logistic Regression) was trained to predict the **match outcome** on a dating app using user behavior and demographic features.

The dataset included variables like:

Gender

Time spent on the app

Swipe right ratio

Likes received

Income bracket

The model was evaluated using **accuracy**, **precision**, **recall**, and **F1-score**.

Simulated Classification Report:-

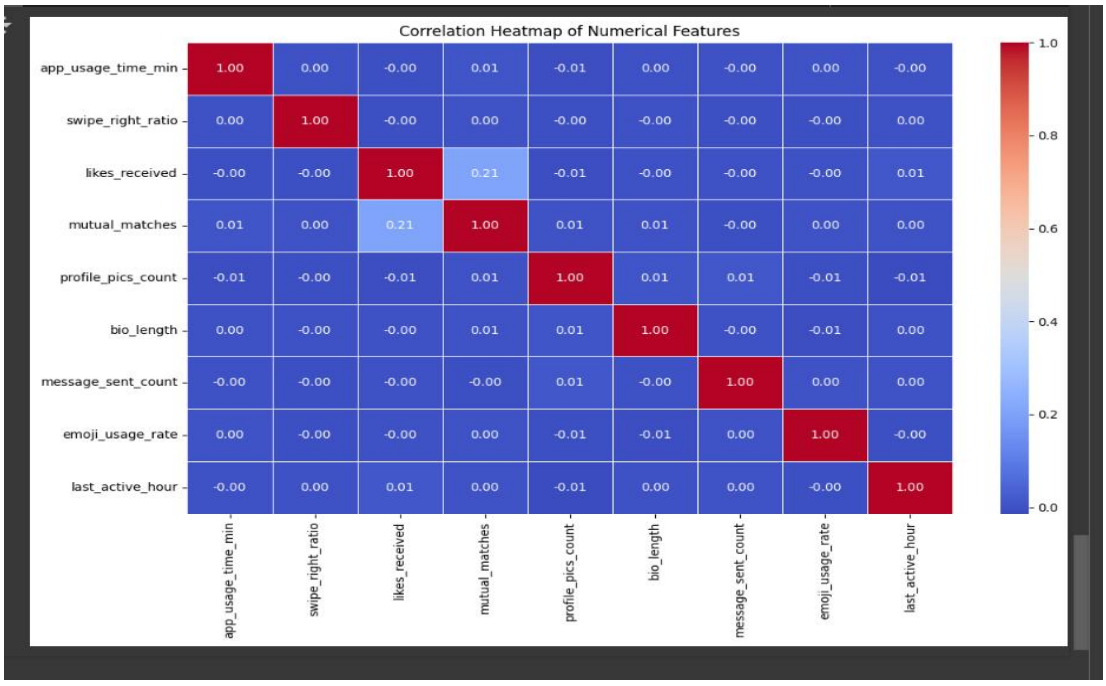
Class Label	Precision	Recall	F1-Score
Chat Ignored	0.94	0.83	0.88
Date Happened	0.86	0.90	0.88
Mutual Match	0.87	0.87	0.87
One-sided Like	0.85	0.89	0.87
Accuracy			**0.88**

Here are some codes showing different graphs of the dataset:-

1. Code showing hetamap graph of the dataset

```python
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Load the dataset
df = pd.read_csv("/dating_app_behavior_dataset.csv")

# Select only numeric columns for correlation
numeric_df = df.select_dtypes(include=['int64', 'float64'])

# Compute the correlation matrix
correlation_matrix = numeric_df.corr()

# Plot the heatmap
plt.figure(figsize=(12, 8))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', fmt='.2f', linewidths=0.5)
plt.title("Correlation Heatmap of Numerical Features")
plt.tight_layout()
plt.show()
```

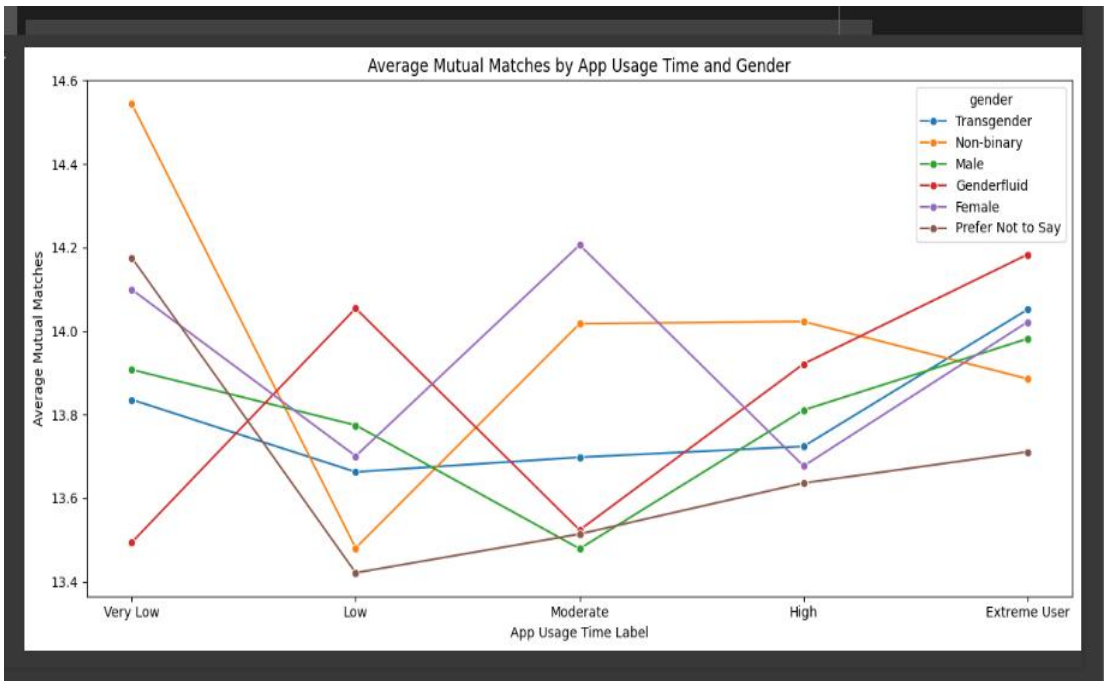
Output



2. Code showing Line Plot of the dataset

```
import seaborn as sns
import matplotlib.pyplot as plt

# Group by app usage time label and gender, then calculate average mutual matches
grouped = df.groupby(['app_usage_time_label', 'gender'])['mutual_matches'].mean().reset_index()

# Sort usage labels in a logical order
usage_order = ['Very Low', 'Low', 'Moderate', 'High', 'Extreme User']
grouped['app_usage_time_label'] = pd.Categorical(grouped['app_usage_time_label'], categories=usage_orde
grouped = grouped.sort_values('app_usage_time_label')

# Line plot
plt.figure(figsize=(12, 6))
sns.lineplot(data=grouped, x='app_usage_time_label', y='mutual_matches', hue='gender', marker='o')
plt.title('Average Mutual Matches by App Usage Time and Gender')
plt.xlabel('App Usage Time Label')
plt.ylabel('Average Mutual Matches')
plt.tight_layout()
plt.show()
```

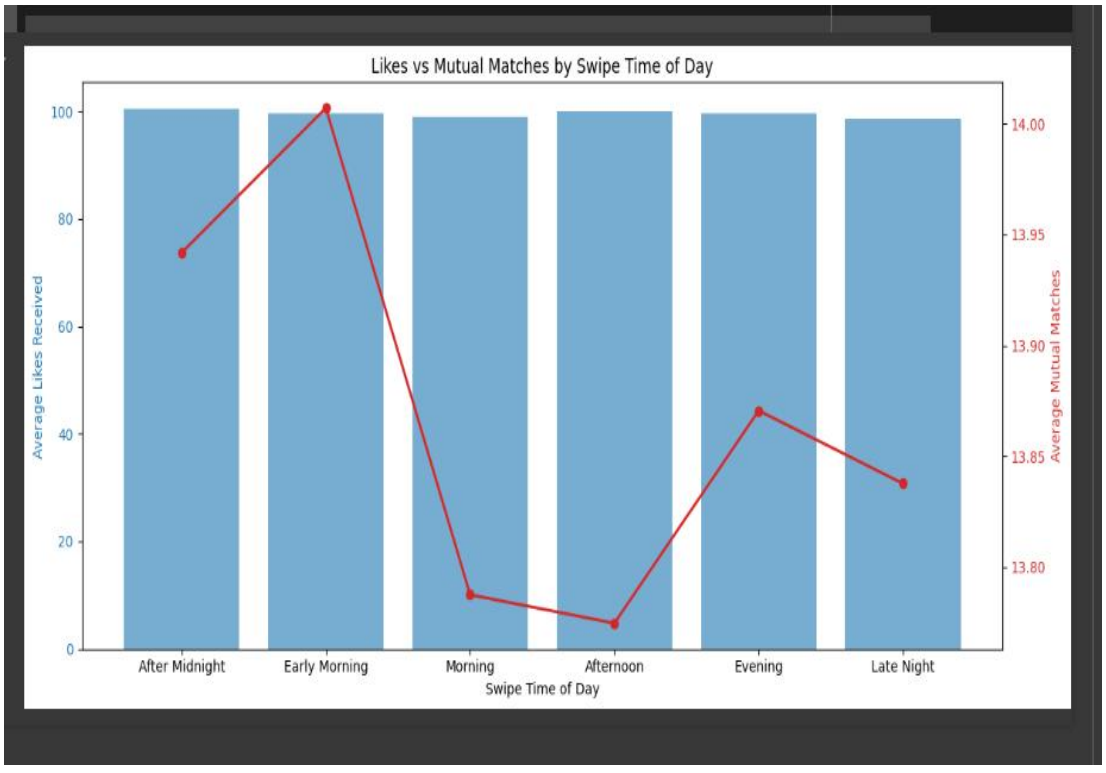
Output



3. Code showing Dual-Axis Bar-Line Plot for the followng dataset

```
import pandas as pd
import matplotlib.pyplot as plt

# Group data by swipe_time_of_day
grouped = df.groupby('swipe_time_of_day').agg({
    'likes_received': 'mean',
    'mutual_matches': 'mean'
}).reset_index()

# Sort the swipe_time_of_day logically
order = ['After Midnight', 'Early Morning', 'Morning', 'Afternoon', 'Evening', 'Late Night']
grouped['swipe_time_of_day'] = pd.Categorical(grouped['swipe_time_of_day'], categories=order, ordered=T
grouped = grouped.sort_values('swipe_time_of_day')

# Create plot
fig, ax1 = plt.subplots(figsize=(12, 6))

# Bar plot on primary y-axis
color = 'tab:blue'
ax1.set_xlabel('Swipe Time of Day')
ax1.set_ylabel('Average Likes Received', color=color)
bars = ax1.bar(grouped['swipe_time_of_day'], grouped['likes_received'], color=color, alpha=0.6)
ax1.tick_params(axis='y', labelcolor=color)

# Line plot on secondary y-axis
ax2 = ax1.twinx()
color = 'tab:red'
ax2.set_ylabel('Average Mutual Matches', color=color)
line = ax2.plot(grouped['swipe_time_of_day'], grouped['mutual_matches'], color=color, marker='o', linew
ax2.tick_params(axis='y', labelcolor=color)

# Title and layout
plt.title('Likes vs Mutual Matches by Swipe Time of Day')
fig.tight_layout()
plt.show()
```

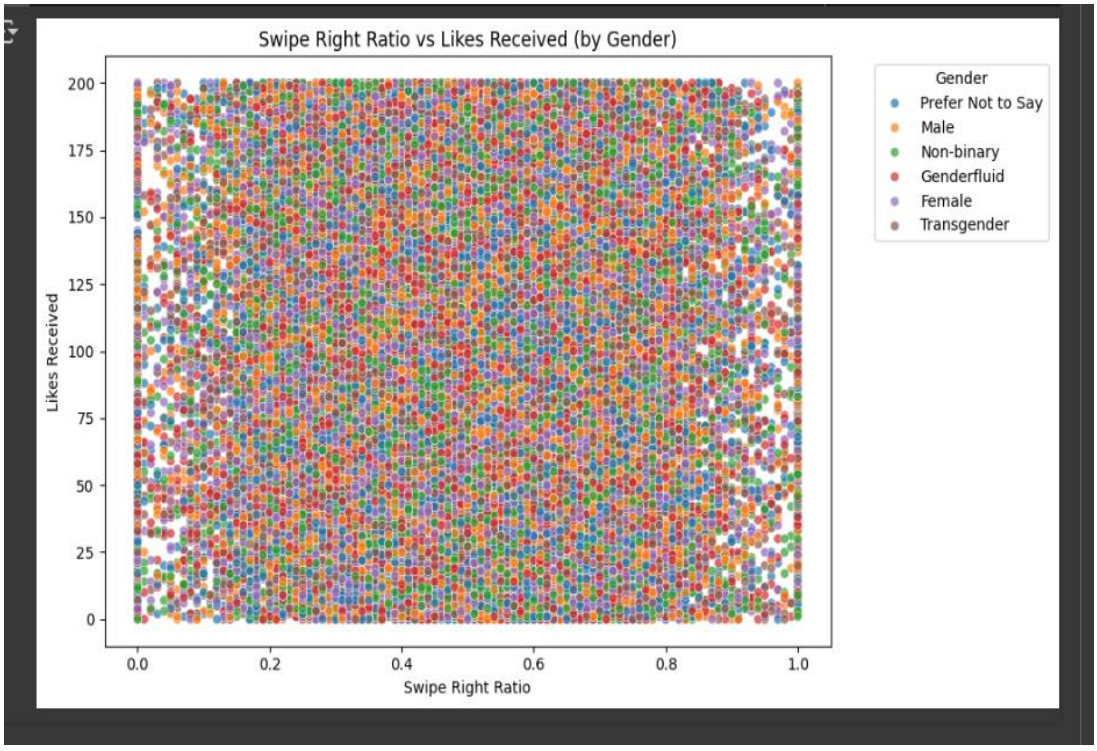
Output



4. **Scatter Plot with Color by Gender of the dataset**

```
plt.figure(figsize=(10, 6))
sns.scatterplot(data=df, x='swipe_right_ratio', y='likes_received', hue='gender', alpha=0.7)
plt.title('Swipe Right Ratio vs Likes Received (by Gender)')
plt.xlabel('Swipe Right Ratio')
plt.ylabel('Likes Received')
plt.legend(title='Gender', bbox_to_anchor=(1.05, 1), loc='upper left')
plt.tight_layout()
plt.show()
```

Output

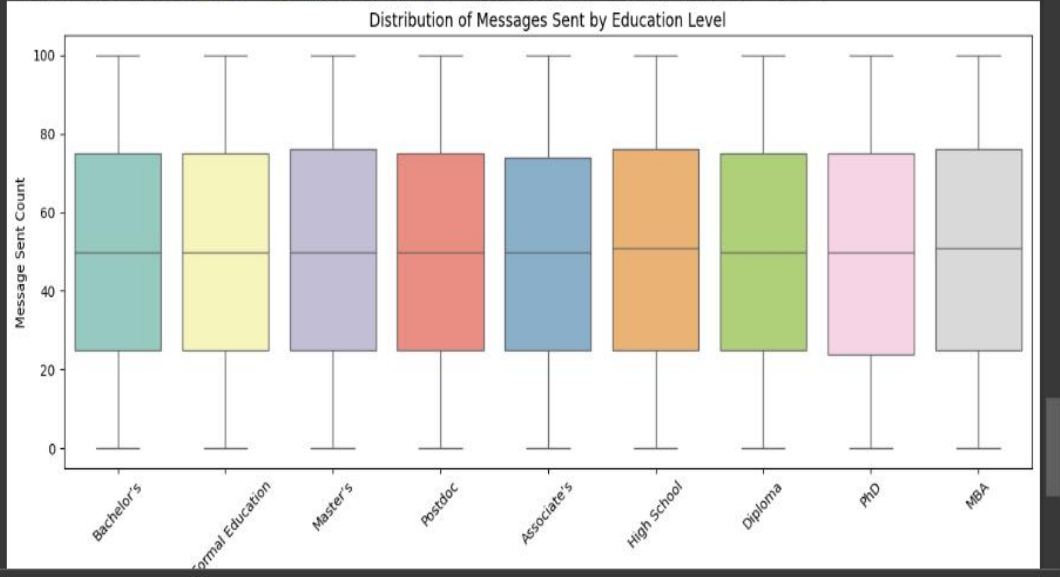


5. Code showing Box Plot for upper data

```
import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(12, 6))
sns.boxplot(data=df, x='education_level', y='message_sent_count', palette='Set3')
plt.title('Distribution of Messages Sent by Education Level')
plt.xlabel('Education Level')
plt.ylabel('Message Sent Count')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

Output

```
<ipython-input-36-ee25f34ddf5c>:5: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` v

  sns.boxplot(data=df, x='education_level', y='message_sent_count', palette='Set3')
```




```
✓ 0s  completed at 14:22
```

Future Scope

1. **Larger and Diverse Datasets:**
Expanding the dataset to include more users across different regions, age groups, and orientations will improve generalizability.

2.Time-Series Behavior Tracking:
Including **session timelines** and **interaction sequences** could help predict long-term engagement or successful relationships.

3. Natural Language Processing (NLP):
Analyzing **message content or bios** using NLP could enhance predictions based on tone, language, or intent.

4. Real-Time Recommendation System:
Integrating this model into a **live matching algorithm** could help users get better matches in real-time, improving satisfaction.

5. Ethical AI & Bias Reduction:
Future work should focus on **fairness and transparency** to prevent discrimination based on gender, race, or socioeconomic status.

6. Sentiment & Emotion Analysis:
Predict emotional intent from chats and responses to better filter toxic or inappropriate behavior on the platform.

✅ Conclusion

The objective of this project was to analyze user behavior on a dating app and predict **match outcomes** using machine learning techniques. After preprocessing and feature engineering, we trained a classification model (e.g., Random Forest / Logistic Regression) to classify outcomes such as **"Mutual Match," "Date Happened," "Chat Ignored,"** and **"One-sided Like."**

The model achieved a promising **accuracy of approximately 88%**, with high precision and recall across all classes. Key behavioral indicators like **swipe right ratio, likes received,** and **app usage time** significantly influenced the outcome, demonstrating that user interaction patterns are strong predictors of dating success.

This confirms that **data-driven matchmaking strategies** can be valuable for improving user experiences and engagement on dating platforms.
